Dockerfile Understanding Projects

PROJECT : 1 Nodejs Application

Objective

To containerize a simple Node.js application using Docker, build an image, run a container, and troubleshoot runtime errors.

Project Structure

```
docker-node-app/
 ├── app.js
 ├── package.json
 └── Dockerfile
```

Step 1: Application Code (app.js)

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.end("Hello from Docker!");
});

server.listen(3000, "0.0.0.0", () => {
  console.log("Server running on port 3000");
});
```

Step 2: Dockerfile

```
FROM node:18
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Step 3: Build Image

docker build -t node-app .

Step 4: Run Container

docker run -p 3000:3000 node-app

PROJECT : 2 Static Website Deployment using Nginx

1. Objective

The objective of this project is to understand the fundamentals of Docker by containerizing and deploying a simple static website using the Nginx web server.

2. Project Overview

In this project, we create a basic HTML webpage, define a Dockerfile, build a Docker image, and run a container to serve the webpage using Nginx.

3. Project Structure

```
nginx-docker-app/
│
├── index.html
└── Dockerfile
```

4. Technologies Used

Docker, Nginx, HTML

5. HTML Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Docker Demo</title>
</head>
<body>
  <h1>Hello from Docker 🚀 </h1>
  <p>This is my first Docker project!</p>
</body>
</html>
```

6. Dockerfile

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
EXPOSE 80
```

7. Build Command

```
docker build -t my-nginx-app .
```

8. Run Command

```
docker run -p 8080:80 my-nginx-app
```

9. Output

Open http://localhost:8080 in your browser.

Project 3: Custom Nginx Page (Upgrade Your Current One)

You already did basic Nginx—now customize behavior.

New Concept

- Overriding default config
- Working with config files inside containers

Structure

```
nginx-custom/
├── index.html
├── nginx.conf
└── Dockerfile
```

---

nginx.conf

```
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
```

```
        index index.html;
    }
}
```

Dockerfile

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
```

Project 4: Python Script Runner (Very Important Concept)

New Concept

- Containers for **jobs (not servers)**
- Understanding `CMD` deeply

Structure

```
python-script/
├── script.py
└── Dockerfile
```

---

script.py

```
print("Hello! This is running inside Docker.")
```

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app
```

```
COPY script.py .

CMD ["python", "script.py"]
```

Run

```
docker build -t python-script .
docker run python-script
```

Project 5:

1.Objective

To understand and practically implement all major Dockerfile instructions by building a real-world Node.js application container.

2. Project Structure

```
docker-full-demo/
├── app.js
├── package.json
├── data.txt
├── start.sh
└── Dockerfile
```

3. Application Code (app.js)

```
const http = require("http");
const fs = require("fs");

const PORT = process.env.PORT || 3000;
```

```
const MESSAGE = process.env.MESSAGE || "Default Message";

const data = fs.readFileSync("./data.txt", "utf-8");

const server = http.createServer((req, res) => {
  res.write(`Message: ${MESSAGE}\n`);
  res.write(`File Data: ${data}`);
  res.end();
});

server.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

4. package.json

```
{
 "name": "docker-full-demo",
 "version": "1.0.0",
 "main": "app.js"
}
```

5. data.txt

This file was added using ADD instruction.

6. start.sh

```
#!/bin/sh
echo "Starting application..."
exec "$@"
```

7. Dockerfile

```
FROM node:18-alpine
ARG APP_VERSION=1.0
ENV PORT=3000
ENV MESSAGE="Hello from Dockerfile"
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
ADD data.txt /app/data.txt
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
```

```
RUN chown -R appuser:appgroup /app
USER appuser
EXPOSE 3000
ENTRYPOINT ["sh", "start.sh"]
CMD ["node", "app.js"]
```

8. Build and Run Commands

```
docker build -t full-docker-demo .
docker run -p 3000:3000 full-docker-demo
```

9. Dockerfile Instruction Explanation

FROM: Base image
ARG: Build-time variable
ENV: Runtime environment variables
WORKDIR: Working directory inside container
COPY: Copy files from host to container
RUN: Execute commands during build
ADD: Advanced copy (supports extraction)
USER: Run container as non-root user
EXPOSE: Document container port
ENTRYPOINT: Fixed executable
CMD: Default command
